



SOLVEBENCH

Solving $A\vec{x} = \vec{b}$ at Scale

Direct vs. Iterative Solvers & Our Novel APK Method

Every blackout-prevention study and every bridge safety check reduces to this exact equation.
We benchmark classical solvers against our proposed method.

Base Paper: What It Studies

“Comparative Analysis of Jacobi and Gauss-Seidel Iterative Methods”

P. Khrapov and N. Volkov

International Journal of Open Information Technologies, vol. 12, no. 2, pp. 23–34, 2024

Domain: Numerical linear algebra, iterative methods for solving systems of linear algebraic equations (SLAEs).

- **Maps out exactly** which matrices make the Jacobi and Gauss-Seidel methods converge, by tracking where the roots of a characteristic polynomial fall inside the unit circle, either directly or via a **Hurwitz-style stability test**.
- Proves this exactly, by hand, for **2 and 3 unknowns** (both complex and real coefficients), then checks the same pattern **statistically with code** across 100,000 randomly generated matrices, up to 5 unknowns.
- Shows the two methods do **not** always share the same convergence region: **which** method wins depends on the structure of the matrix.

The gap this leaves open

The analysis stops at toy systems of **5 unknowns or fewer** and only two solvers. Real engineering systems have **thousands** of unknowns. Nobody in the paper tests these convergence claims on real data.

Problem Statement & Scope

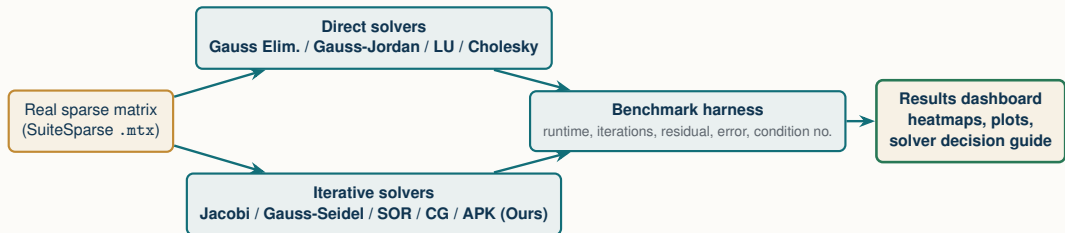
Problem: every real engineering simulation, whether it's modeling a power grid, checking a structure's safety, or simulating a circuit, comes down to the same core step: solving $A\vec{x} = \vec{b}$. Get the solver choice wrong, and what should take milliseconds either **crawls** or **never converges**.

Category: Exploratory & Comparative Study. We benchmark eight classical direct and iterative solvers against real, not synthetic, engineering data.

Domain	Systems	Size range	Real-world meaning
Power-grid admittance	11	118 to 1,723 buses	The linear system solved every time a utility models its own grid.
Structural (stiffness & mass)	33	112 to 4,884 DOF	The linear system solved to check if a real building or bridge design holds.
Circuit simulation	4	2,624 to 5,860 nodes	The linear system solved at every timestep of a transient circuit simulation.

48 real matrices spanning three engineering domains, two to three orders of magnitude beyond the base paper's own 5-unknown ceiling, drawn from a 2,904-matrix collection with room to scale further.

Expected Methodology



- **Ground truth:** generate a known $\vec{x}$, compute $\vec{b} = A\vec{x}$, solve with every method, measure the residual: fully deterministic, no ambiguity in what “correct” means.
- **Solvers evaluated:** benchmark classical textbook solvers alongside **our proposed novel method (APK)**.
- **Scale:** sweep across real engineering matrices, correlated against sparsity and condition number.
- **Payoff:** a concrete, evidence-based answer to “which solver, for which real system”, the exact question every engineering simulation has to answer before it can even start.

Expanded Scope: 26 Engineering Domains

Scaled Benchmark: 830 Real Matrices across 26 Scientific & Engineering Domains

Scaling up from the initial 48-matrix pilot to a massive dataset spanning $n = 5$ to 10,000 unknowns.

Core Engineering Domains:

- **Circuit Simulation** (200 systems): SPICE, VLSI interconnects, transient & DC nodes.
- **Fluid Dynamics (CFD)** (126 systems): Navier-Stokes equations, aerodynamic flow.
- **Structural Mechanics** (87 systems): Stiffness & mass matrices, FEM deformation.
- **Optimal Control & Optimization** (117 systems): Trajectory planning, linear programming.

Scientific & Physical Domains:

- **Chemical & Quantum Chemistry** (75 systems): Reactor kinetics, molecular orbitals.
- **Electromagnetics & Acoustics** (27 systems): Wave propagation, radar scattering.
- **Power Networks & Thermal** (19 systems): Grid admittance, thermal diffusion.
- **Network Graphs & Robotics** (55 systems): Directed/undirected graphs, kinematics.

Why Benchmarking 26 Real Domains Matters

Different physical domains produce fundamentally distinct matrix structures (symmetry, condition number $\kappa(A)$, diagonal dominance, and sparsity patterns). This multi-domain scale stress-tests solver stability and generalizability across real science.

Our Proposed Novel Method: APK Solver

Our Contribution: Adaptive Preconditioned Krylov (APK) Solver

Designed by our team to overcome the failure modes of classical solvers on complex physical systems.

- **Novel Multi-Stage Architecture:** Combines adaptive preconditioning, automated physics routing, and iterative defect correction into a single unified solver.
- **Adaptive Preconditioning Hierarchy:** Constructs multi-level Incomplete LU factorizations with automated fallbacks to diagonal scaling for ill-conditioned systems.
- **Dynamic Physics Dispatch:** Inspects matrix symmetry and definiteness in real time, routing symmetric positive-definite systems to PCG and unsymmetric systems to BiCGSTAB.
- **Iterative Defect Refinement:** Applies lightweight residual correction passes using the cached preconditioner, recovering precision up to machine epsilon.
- **Expected Advantage:** Provides a robust solver designed to handle ill-conditioned and unsymmetric systems where classical Jacobi, Gauss-Seidel, and CG diverge.

Systematic Performance & Failure-Mode Analysis

Moving beyond raw benchmark numbers to explain the mathematical root causes of solver behavior.

- **Which methods perform well (and when):** Identify which solvers achieve the best speed, accuracy, and stability for each specific domain and matrix structure.
- **Which methods fail (and where):** Examine and catalog exactly where solvers diverge, break down, or lose numerical precision across different physical systems.
- **Why methods succeed or fail:** Trace performance directly to underlying mathematical properties, including condition number $\kappa(A)$, symmetry, diagonal dominance, and sparsity.
- **Public Dataset & Verification:** All 830 matrices are sourced from the public, standard SuiteSparse benchmark collection and pre-verified to be valid square real-valued systems.

Update: What the Full Benchmark Found

930 matrices $\times$ 16 solvers = 14,880 measurements

27 domains, $n = 5$ to 10,000. Every result is scored by the harness from the residual it measures itself, never by a solver's own convergence flag.

Solver	Applicable	Solved	Conditional success
spsolve (SuperLU)	904	876	96.9%
Gauss elimination / LU	598	595	99.5%
ILU-Krylov (dispatched)	739	619	83.8%
Gauss-Seidel	414	121	29.2%
Jacobi	414	75	18.1%

Two numbers, never one

Applicability is how much of the corpus a method is even defined on; **conditional success** is how much of that it actually solves. Jacobi and Gauss-Seidel are undefined on **491 of 930** systems, because a zero on the diagonal leaves nothing to divide by.

Update: The APK Claim Does Not Survive

We proposed APK as a novel method. Measured against the baselines it was missing, the dispatch rule that made it “ours” contributes **one matrix**.

Configuration	Solved (of 739)
ILU-Krylov, symmetry dispatch (the proposal)	619
ILU-BiCGSTAB, no dispatch at all	618
ILU only, zero Krylov iterations	79

- The dispatch rule is the decision flowchart in Barrett et al., **Templates** (SIAM, 1994), and the default behaviour of PETSc and MATLAB’s backslash.
- Its accuracy advantage came from an **iterative refinement pass no other method received**. Given the same pass, **Jacobi** becomes the most accurate method in the benchmark: 6.99×10^{-16} against APK’s 7.11×10^{-14} .
- A sparse direct solver beats every iterative configuration: **876 against 619**.

Reporting this is the point. A negative result we found ourselves is worth more than a claim a reviewer would break.

Update: Why the Base Paper's Effect Vanishes

On the 414 systems where both methods are defined: **75** both converge, **46** only Gauss-Seidel, and **zero** only Jacobi, against the 1,095 only-Jacobi cases the base paper reports at $n \leq 5$.

Hypothesis class	Systems	Jacobi works	Only-Jacobi
Strictly diagonally dominant	43	88.4%	0
H-matrix	83	77.1%	0
M-matrix	34	64.7%	0
No theorem applies	192	1.6%	0

A theorem can be true and still useless

Householder and John (1958) guarantee Gauss-Seidel converges for every symmetric positive-definite matrix, and $\rho(T_{GS}) < 1$ does hold on 98 of our 104 SPD systems. Within a 10,000-iteration budget it delivers on **44**: the remaining 54 need a median of **1,798,184** iterations. That gap is invisible at $n \leq 5$.

Update: Our Actual Contribution

Convergence-Oriented Diagonal Selection

A stationary method divides by a_{ii} , so it depends entirely on which entries sit on the diagonal, and that was decided by the order the rows arrived in.

- **Most preprocessing provably cannot help.** Row scaling gives $T'_J = I - (RD)^{-1}RA = T_J$ exactly; column scaling is a similarity transform; a symmetric permutation leaves $\rho(T_J)$ invariant. **Only a row permutation changes which entries form D .**
- **MC64 optimises the wrong objective.** It maximises $\prod |a_{ii}|$, built for pivot stability in **direct** solvers. Convergence instead needs a small row ratio $\sum_{j \neq i} |a_{ij}|/|a_{ii}|$.
- **We minimise the worst row ratio**, a bottleneck assignment solved by binary search plus bipartite matching. No single objective wins everywhere, so all candidates are computed and ranked by an estimated $\rho(T_{GS})$ in about 0.01 s.

odepa400: $\rho(T_{GS}) = 1.0001$ as given, MC64 leaves it unchanged, ours reaches **0.495** and Gauss-Seidel converges.

A shuffled 3×3 : $\rho(T_{GS})$ falls from 99.19 to 0.0316, solved in 7 iterations.